

How Full and What Is Wrong: A Fog-to-Cloud Monitor for a Two-Lot Smart Parking Estate

Pooja

Student ID: X25120921

MSc in Cloud Computing

Fog and Edge Computing (H9FECC)

National College of Ireland

x25120921@student.ncirl.ie

Abstract—An operator watching a parking estate is really asking two questions at once, and they are not the same question. One is how full each lot is, a smooth quantity that climbs and falls with the day. The other is whether anything is wrong, a barrier stuck open, an inflow surge, a car that has sat far too long, which has nothing to do with how full the lot happens to be. This project answers both from the same stream and keeps them apart on purpose. Ten sensor processes drive five signals, occupied spaces, entry rate, exit rate, average dwell time, and gate-fault events, across two lots; a fog node on the edge host windows each lot-and-signal stream, reduces it to a five-number summary, and raises the hard fault alarms locally. Behind the edge, a serverless backend on Amazon SQS, AWS Lambda, and Amazon DynamoDB persists every window, and a dashboard served through Amazon API Gateway derives, fresh on every read, a graded occupancy tier from each lot's fullness, then fuses it with the fault alarms into one status badge where any active fault outranks the fullness tier, so an operator sees the worse of the two at a glance. The occupancy tier is never stored; it is computed on read from the same windows. The system was provisioned to a real Amazon Web Services account rather than a local emulator; 127 automated tests pass, and verification against the live endpoint confirmed both lots streaming all five signals, one lot holding a normal tier while the other raised a gate-fault alarm, a live occupancy trend, and a healthy pipeline, with the frontend delivered from Amazon S3.

Keywords—smart parking, occupancy monitoring, intelligent transportation systems, fog computing, edge computing, Internet of Things, serverless computing, Amazon SQS, AWS Lambda, Amazon DynamoDB

I. INTRODUCTION

Parking is one of the most quietly expensive problems in a city: drivers circling for a free bay waste fuel and time and add to congestion, while an operator with no live view of the estate cannot steer them or spot a barrier that has failed. Sensing a lot is by now well understood, and surveys of smart-parking systems catalogue the occupancy sensors, guidance signs, and networks that make an estate legible [1], [2]. The harder question is what to do with the readings once they exist, and in particular where the readings are turned into a decision.

The Internet of Things makes it cheap to instrument every bay and gate in a lot [3], but streaming every raw sample from every lot to a distant cloud region to be judged there is wasteful in two directions at once. It spends bandwidth continuously on readings that, taken singly, rarely change anything, and it

defers the one moment that matters, a signal crossing into its danger range, until after a network round trip. Fog computing was proposed to close that gap by placing a compute tier between the sensors and the cloud, near enough to the sources to summarise, filter, and evaluate rules before anything travels [4], [5], and it is one expression of the broader movement of computation back toward the network edge [6], [7]. Behind that edge tier, the cloud supplies the elastic, pay-as-used model that removes the need to size a server for a peak that sits idle most of the day [8].

This project builds that two-tier design for a scaled but faithful slice of a monitored estate: two lots, designated lot-a and lot-b, each carrying the same five signals, run as ten independent sensor processes. What gives the design its own character is a deliberate separation of two things an operator conflates at their peril. Fullness is a graded quantity: a lot is somewhere on a scale from empty to full, and the useful reading of it is a tier, quiet, busy, or nearly full. A fault is a different kind of event: a gate stuck faulting, an inflow surge, a car dwelling far beyond a normal stay. The two are largely independent, a lot can be busy but perfectly healthy, or nearly empty yet faulted, and the monitor keeps them on separate axes. The fog node raises the hard fault alarms, and the dashboard derives the fullness tier on read, then the two are fused into a single badge in which any active fault outranks the tier, so what an operator sees is always the more urgent of the two.

The work pursues four objectives, each demonstrated on a running system rather than only described. The first is a sensor and fog layer that genuinely windows and evaluates five heterogeneous signals at sampling and dispatch rates that are configurable rather than fixed in code. The second is a backend that scales through managed, elastic AWS primitives instead of a single always-on server. The third is a dashboard that turns stored windows into an operational reading, in particular the graded occupancy tier fused with the fault alarms. The fourth, and the one that separates a working system from a plausible one, is deployment onto a real public-cloud account and verification against the live endpoint. Scope is held to these layers at a two-lot scale rather than a whole city's estate. The remainder of this paper is organised as follows. Section II presents the architecture and the reasoning behind each choice,

including the alternatives weighed and the security posture. Section III covers the implementation, the automated tests, continuous integration, the real deployment, and the evidence gathered from the running system. Section IV closes with a critical assessment of the trade-offs, the limits, and where the work could go next.

II. SYSTEM ARCHITECTURE AND DESIGN

Fig. 1 shows the deployed system end to end: ten sensor processes and the fog node sharing one edge host, a managed queue, two independently deployed functions, a key-value store, and a browser-facing bucket reached through a managed gateway.

A. Sensor and Fog Layer

Each of the ten sensor processes represents one signal on one lot. On every sampling tick a process advances a bounded random walk: it nudges its previous value by a step drawn uniformly from a small symmetric range and clamps the result into a physically plausible band for that signal, so consecutive readings stay close together like a real field sensor rather than independent noise. The step sizes are set per signal to match how the real quantity moves, so an occupancy count that fills and empties over minutes drifts differently from a dwell time measured in tens of minutes. Each process runs sampling and dispatch as two coroutines on a single event loop, coordinated by one lock: the sampler appends a reading under the lock, and the dispatcher, on its own slower cadence, takes the lock only long enough to swap the accumulated buffer out for an empty one before shipping it, so a sample can never be lost to a dispatch in progress and neither cadence blocks the other. The sampling interval defaults to two seconds and the dispatch interval to ten, and keeping them separate lets a fast-moving quantity be sampled often while still being shipped in economical batches. Each dispatch is a compact JSON object naming the signal, the lot, and the unit and carrying the batch of values gathered since the last dispatch, which at the default rates is roughly five readings amounting to a few hundred bytes on the wire, well under a kilobyte. The process posts it over HTTP to the fog node's ingest endpoint, and if that post fails because the fog node is briefly unreachable, the batch is prepended back onto the buffer and retried on the next dispatch, ahead of anything sampled in the meantime, so a transient outage delays readings in arrival order instead of dropping or reordering them.

The fog node is a Python service built directly on the standard library's own WSGI server, running as its own container on the edge host distinct from the sensor processes and the cloud functions, with a small hand-written route table rather than a web framework. Every ingest request is validated before anything is buffered, so a batch that is missing a field, carries a non-numeric value, or is not a JSON object is rejected with a clear error rather than corrupting a window. The buffering itself is worth attention, because an edge host has finite memory and a monitoring workload runs indefinitely. Each lot-and-signal stream accumulates into a bounded ring

buffer that keeps only the most recent readings and silently drops the oldest once it is full, so a fog node that briefly cannot flush, or a sensor that dispatches faster than expected, can never grow the buffer without limit; the window is a fixed recent slice, not an unbounded backlog. When the window boundary arrives on a fixed timer, every non-empty group is snapshotted and the buffers are cleared in one step so a fresh empty window is in place immediately, and the summarising happens on the snapshot outside that step.

When a window closes, each non-empty lot-and-signal group is reduced to a summary carrying the count, minimum, maximum, average, and latest value, and that summary, not any raw reading, is evaluated against the hard fault rules. Those rules are a small table of predicates, each owning the signal it applies to and the field of the summary it reads: an average occupancy above two hundred and seventy of three hundred bays raises a near-full alarm, an average entry rate above twenty vehicles a minute raises an inflow-surge alarm, an average dwell time above three hundred minutes raises a long-stay alarm, and, unlike the others, a maximum gate-fault count above three raises a gate-fault alarm. The gate-fault rule reads the window's peak on purpose, because a gate fault is a discrete event and a single spike within a window is the thing worth reporting, where an average would dilute it away. The exit rate deliberately carries no rule; it is stored and shown as context, the natural counterpart to the entry rate, but a lot emptying is not a fault. The fog node also publishes this rule set, as a descriptive table naming each signal's field, operator, and limit, on a read-only endpoint, so the dashboard can show operators the thresholds in force without hard-coding them a second time.

The fog node's output interface is a real managed queue rather than a direct write to storage. On each window boundary it batches the closed summaries and sends them to Amazon SQS in grouped calls, each carrying up to the service's per-call limit of ten summaries, so a whole flush costs a number of send operations proportional to the summaries closing divided by ten rather than one send per summary. The queue's address is resolved once and reused, retried while the queue is still being provisioned because on a fresh deployment it may not exist when the edge host boots, and a failed flush is logged while the node keeps running rather than crashing, a bias toward keeping the edge live that suits a monitoring workload where the next window is only seconds away.

B. Backend Layer

The cloud tier follows a single ingest path with a branch for the operator interface. A queue receives the fog node's summaries, one function drains the queue and writes to storage, and a second, separately deployed function reads that storage back to answer the dashboard. Splitting the write path from the read path is deliberate: ingestion and querying then scale and fail independently, so a surge of windows cannot slow the dashboard and a burst of operator activity cannot back up ingestion.

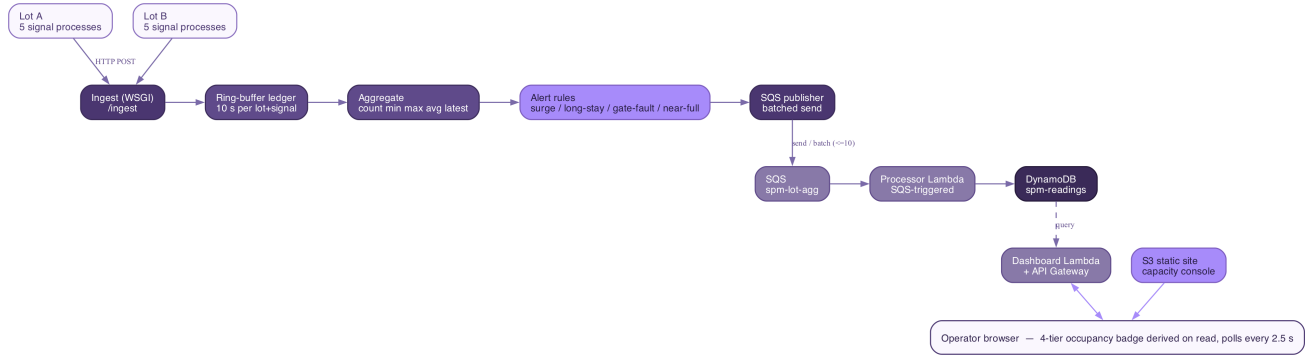


Fig. 1. Deployment topology. Ten sensor processes, two lots of five signals each, and the fog node share one Amazon EC2 edge host. The ingest path runs from the fog node through Amazon SQS and the processor function into the Amazon DynamoDB table; the serving path runs from the dashboard function’s read of the stored windows out through Amazon API Gateway to the browser, with static assets served from Amazon S3. The fog node reduces each window to a five-number summary and raises the hard fault alarms before anything is sent. The graded four-tier occupancy badge is derived at read time in the browser-facing function, not stored. The dashed edge is the dashboard function’s query back over the stored windows.

Queue instead of a direct call. The fog node could invoke the storage function directly and synchronously, dropping the queue and saving a hop. That option was declined. A synchronous call binds the edge to the availability and latency of the cloud function: throttle it, leave it cold, or take it down briefly, and the fog node stalls or sheds data as the back-pressure travels back toward the sensors. A queue severs that coupling, absorbs bursts, retries on the consumer’s behalf, and lets the fog node treat a summary as delivered the moment it is enqueued [9]. On a ten-second cadence the small added latency is immaterial, while the decoupling and durability are exactly what an intermittent edge link needs.

Event-driven functions instead of a standing server. Both the ingestion and the interface tiers run as event-driven functions. This fits work that arrives in bursts and, for the interface, only intermittently: billing is per request rather than for idle capacity, and the platform adds and removes instances as demand shifts, with nothing reserved in advance [8]. The recognised cost is the cold start after a period of idleness [10]; its effect here is small, because steady queue traffic keeps the ingestion function warm and the dashboard’s own polling keeps the interface function warm. The processor function is triggered by the queue, holds no state between invocations, and is billed only for the work it does.

Key-value store instead of a relational database. A relational engine would offer joins and open-ended queries, neither of which this workload uses. Its access pattern is narrow and known in advance: append summaries in time order, then fetch the most recent windows for a given signal. Amazon DynamoDB, an on-demand key-value store built for that mix of heavy writes and key-based reads, holds its performance as the table grows and spares the operational overhead of running a relational server, the balance the original Dynamo work set out and the managed service now carries [11], [12]. The key design follows straight from the query. Each stored window is keyed on its signal as the partition and on a sort value that leads with the window’s closing timestamp and then the lot identifier, so windows for one signal are already

ordered chronologically inside a single partition and the newest are read with one bounded, descending, single-partition query rather than a scan; the lot in the sort tail keeps the two lots’ windows for the same signal from colliding.

C. Serving Layer, the Occupancy Badge, and Security

The dashboard reaches the read function through a managed gateway that publishes a public REST endpoint, and the read function is the same standard-library WSGI application that serves the stack locally, wrapped by a thin adapter that maps a gateway proxy event onto a WSGI call and hands the response back unchanged, so one body of routing code runs in both places. Object storage holds the static frontend, one page, a stylesheet, the page script, and a charting library kept in the repository rather than pulled from a content-delivery network. The function exposes a small, read-only surface: a readings endpoint returning a recent series for one signal, a lots endpoint giving each lot’s latest window per signal together with its derived tier and status, a thresholds endpoint that forwards the fog node’s published rules, a backend-statistics endpoint, and a health endpoint assembled from four live sources, the store for the freshest window, the queue, the processor function’s deployment state, and the fog node’s reachability, returning a pipeline flag that is true only when a window actually reached the store recently.

The graded occupancy tier is the piece of interpretation the hard alarms do not provide, and it is computed here, on every read, rather than stored. For each lot the dashboard takes the latest occupancy window, expresses it as a percentage of that lot’s capacity, and buckets it: below three-quarters full is quiet, three-quarters or more is busy, and nine-tenths or more is nearly full. That tier answers the fullness question alone. The fault question is answered separately, by the hard alarms the fog node already raised and stored on each window. The two are then fused into the single badge an operator reads: if any of the lot’s latest windows carries a fault alarm, the badge shows alert and outranks whatever the fullness tier would have said, because a faulted lot needs attention whether it is

empty or full; otherwise the badge shows the fullness tier. Keeping the tier a read-time derivation rather than a stored field means the stored windows stay the single source of truth, the tiering thresholds can be retuned without rewriting anything already persisted, and a lot with no occupancy window yet still yields a meaningful pending state rather than a wrong one. On the page, each lot is a card showing its capacity gauge and fused status badge alongside its supporting signals, with an occupancy trend across both lots and any firing alarms named as tags; a fixed browser-side timer re-fetches the lots view, health, and statistics every 2.5 seconds and repaints from whatever the responses hold, so the view stays current without a manual reload and without any server-push channel. The page learns which gateway origin to call from a single placeholder written into it and substituted with the real endpoint when the assets are uploaded; run locally, where one process serves both the page and the interface, the placeholder resolves to empty and requests fall back to the same origin. The security posture is modest and deliberate for a scaled demonstration: the fog node's health and threshold endpoints answer without a credential check because they expose only aggregate window state and configured limits, never a live per-sensor reading, while the cloud tier relies on the managed services' own access control and on scoping each function to the queue, table, and gateway it needs.

III. IMPLEMENTATION

The sensor processes, the fog node, and both cloud functions are written in Python. The fog node and the dashboard are built directly on the standard library's WSGI server with a hand-written route table, so neither carries a web framework; the AWS SDK for Python carries the queue, table, and function calls, and the frontend renders its trend with a locally vendored charting library so the page has no external dependency at load time. The pieces are kept as small single-purpose modules, the validation separated from the buffering, the aggregation from the alarm rules, and the occupancy tier from the storage access, so each is unit-testable on its own without standing up a server or reaching into a live service. The edge tier is orchestrated with Docker Compose, the cloud tier is provisioned with Terraform, and continuous integration runs on GitHub Actions.

A. Testing and Continuous Integration

The system carries an automated suite of 127 tests spread across every component: the bounded-walk sensor and its retain-in-order dispatch, the ingest validation, the bounded ring buffer's accumulate and snapshot, the window aggregation arithmetic, the hard fault rules, the batched publisher, the processor's transform into a stored item, and, on the dashboard side, the occupancy tier and its fault override, the per-lot grouping, the health assembly, the readings query, and the thresholds proxy. They run without any cloud connection by exercising the units against hand-written fake AWS clients, so each test can assert on the exact shape of the request a given call would issue rather than depending on a live service. The rule and tier tests are pointed deliberately at the boundaries,

checking a value one step inside a threshold and one step outside it, so a rule that should fire and a tier boundary that should flip are both pinned down; the fault-override behaviour is tested on its own, so that a lot which is only lightly occupied but carrying a fault is confirmed to read alert rather than quiet. The publisher tests check that a flush is split into batches no larger than the queue's per-call limit, the processor tests check the composite sort value that orders windows within a partition, and the fog ingest path is proven with a real HTTP request against a running local server rather than only a unit test of the validator in isolation, since that request path is what the deployment actually exercises. The suite runs on every push through GitHub Actions, so a change that breaks a threshold, a batch size, a query shape, or the tier logic fails in continuous integration rather than after deployment.

Testing against fakes rather than a live queue was a deliberate choice with a cost worth acknowledging: a fake asserts on the request a call would make, not on how the managed service behaves under load, so the batched-send limit and the queue's own retry semantics were reasoned about from the service documentation [13] and then confirmed against the live deployment rather than in unit tests.

B. Deployment and Live Verification

The cloud tier was provisioned to a real AWS account in a single Terraform apply that created twenty-four resources, with the account confirmed before anything was applied and the working state isolated so the apply could add only its own resources and never disturb anything already present. Describing the whole cloud tier as code, rather than clicking it together once, means the same definition builds the queue, both functions with their triggers and permissions [14], the table, the gateway with its single catch-all route [15], and the two buckets in one reproducible step, and tears them down as cleanly. The edge host runs the fog node and the ten sensor containers under Docker Compose; all the deployed resources carry a common prefix so the deployment is easy to identify and to remove as a unit, and a fixed public address in front of the edge host lets the read function reach the fog node's health and threshold endpoints at the same location across restarts.

Verification was carried out against the live deployment rather than a local emulator, and Fig. 2 is the deployed dashboard during that check. Within about a minute of start-up the pipeline reported healthy end to end: the fog node, the queue, the processor function, and the freshest-window recency all read healthy, and the newest window was a few seconds old on each poll. Both lots rendered all five signals with their current value, and each lot's fused badge summarised it at a glance: one lot sat at a quiet occupancy and read normal, while the other, holding a lower occupancy, nonetheless read alert because its gate-fault count had crossed the peak limit and raised a gate-fault alarm, which is exactly the case the fault override exists for, and the alarm was named as a tag on that lot's card. The occupancy trend showed the two lots crossing over as one filled and the other emptied, the backend-statistics endpoint showed the stored record count

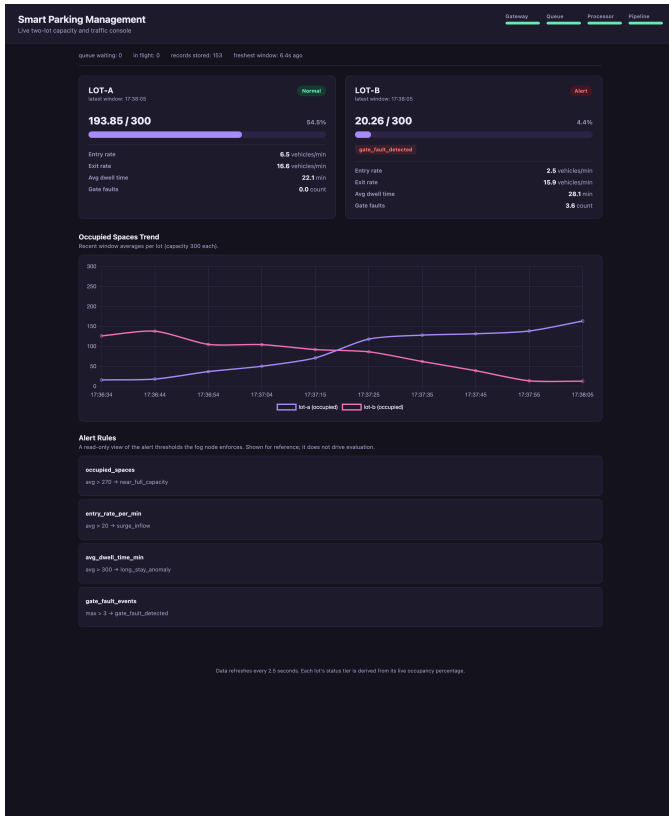


Fig. 2. The deployed dashboard reading from the live endpoint. Each lot shows its capacity gauge and fused status badge alongside its supporting signals; an occupancy trend spans both lots; the header strip shows the four pipeline indicators. Here one lot reads normal while the other reads alert on a gate-fault alarm despite a low occupancy, the fault outranking the fullness tier.

climbing across successive polls rather than a static number, and reaching the dashboard from a browser also exercised the cross-origin path from the storage bucket to the gateway, which returned data cleanly.

IV. CONCLUSION

This project set out to build and, above all, to deploy a fog-to-cloud pipeline that turns five signals across two parking lots into a live operational reading, and it met that goal on a running system rather than on paper. The edge tier does genuine work before anything travels: it windows and reduces each lot-and-signal stream and raises the hard fault alarms locally, so only compact summaries cross the network and a fault is named in the window it appears. The cloud tier scales through decoupling and elasticity rather than a pre-sized server, using a queue to absorb bursts and separate the write and read paths, event-driven functions billed only for the work they do, and an on-demand key-value store whose key design turns the dashboard's one query into a single-partition read. The whole system was provisioned to a real account by one infrastructure-as-code apply and verified end to end against the live endpoint, with 127 automated tests guarding its behaviour and continuous integration running them on every change.

The design's most consequential choice was refusing to collapse two different questions into one number. How full a lot is and whether it is faulted are independent facts, and a monitor that scored them on a single scale would either cry alarm at a merely busy lot or hide a fault behind a low occupancy. Keeping fullness as a graded tier derived on read, keeping faults as hard alarms raised at the edge, and fusing them only at the last step, where a fault outranks the tier, gave an operator a badge that is honest about both at once. The supporting choice was quieter but load-bearing: deriving the tier at read time rather than storing it kept the stored windows as the single source of truth, so the tiering can be retuned, or a lot with missing telemetry handled gracefully, without rewriting anything already recorded.

The work has clear limits. It runs at a two-lot scale with simulated sensors, its fault limits and tier thresholds are fixed in code rather than learned from each lot's own daily and weekly rhythm or adjustable at runtime, and its security posture is intentionally modest for a demonstration. Natural next steps are to widen it to a real estate of instrumented lots and real occupancy feeds, to move the fixed thresholds to a managed configuration an operator could tune per lot without a redeploy, to add authentication and a custom domain in front of the public endpoint, and to retain longer history so a lot's daily fill-and-empty curve can be trended and predicted rather than only read on the latest window. None of these changes the shape of the pipeline, which is the point: the edge-aggregation and cloud-serving design carries from two lots to many without rethinking the tiers.

REFERENCES

- [1] T. Lin, H. Rivano, and F. Le Mouél, "A Survey of Smart Parking Solutions," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 12, pp. 3229–3253, 2017.
- [2] E. Polycarpou, L. Lambrinos, and E. Protopapadakis, "Smart Parking Solutions for Urban Areas," in *Proc. IEEE Int. Symp. World of Wireless, Mobile and Multimedia Networks (WoWMoM)*, 2013, pp. 1–6.
- [3] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A Survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog Computing and Its Role in the Internet of Things," in *Proc. MCC Workshop on Mobile Cloud Computing*, 2012, pp. 13–16.
- [5] A. Yousefpour et al., "All One Needs to Know about Fog Computing and Related Edge Computing Paradigms: A Complete Survey," *Journal of Systems Architecture*, vol. 98, pp. 289–330, 2019.
- [6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [7] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [8] M. Armbrust et al., "A View of Cloud Computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [9] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA: Addison-Wesley, 2003.
- [10] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking Behind the Curtains of Serverless Platforms," in *USENIX Annual Technical Conference*, 2018, pp. 133–146.
- [11] G. DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store," in *Proc. ACM SIGOPS Symp. Operating Systems Principles*, 2007, pp. 205–220.
- [12] M. Elhemali et al., "Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service," in *USENIX Annual Technical Conference*, 2022, pp. 1037–1048.

- [13] Amazon Web Services, “Amazon Simple Queue Service Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/sqs/>
- [14] Amazon Web Services, “AWS Lambda Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/lambda/>
- [15] Amazon Web Services, “Amazon API Gateway Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/apigateway/>
- [16] IEEE, “Manuscript Templates for Conference Proceedings,” 2024. [Online]. Available: <https://www.ieee.org/conferences/publishing/templates.html>

The source code and deployment configuration are available at [GitHub repository URL].